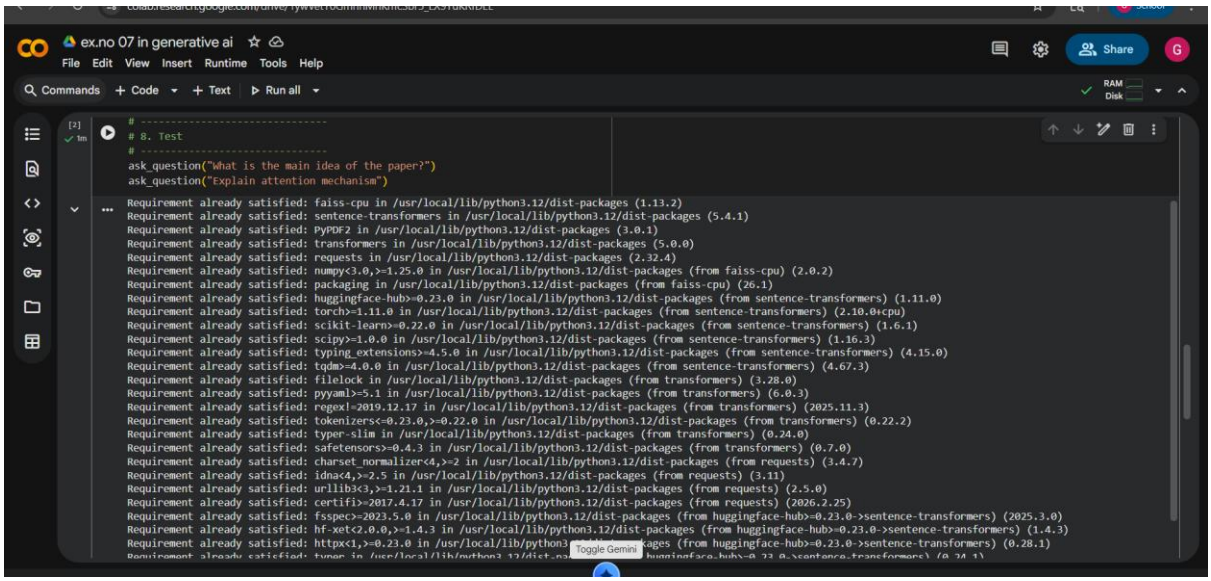
231501053

Gokulakkannan P

Generative AI

ex.no:07



ex.no 07 in generative ai

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```
    reader = PdfReader(file_path)
    text = ""
    for page in reader.pages:
        if page.extract_text():
            text += page.extract_text() + "\n"
    return text

document_text = load_pdf(pdf_path)

# ------------------------------
# 3. Chunk Text
# ------------------------------
def chunk_text(text, chunk_size=300):
    words = text.split()
    return [" ".join(words[i:i+chunk_size]) for i in range(0, len(words), chunk_size)]

chunks = chunk_text(document_text)

# ------------------------------
# 4. Embeddings
# ------------------------------
model = SentenceTransformer('all-MiniLM-L6-v2')
embeddings = model.encode(chunks)
embeddings = np.array(embeddings).astype("float32")

# ------------------------------
# 5. FAISS Index
# ------------------------------
index = faiss.IndexFlatL2(embeddings.shape[1])
index.add(embeddings)
```

ex.no 07 in generative ai

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```
# Install libraries
!pip install faiss-cpu sentence-transformers PyPDF2 transformers requests

import faiss
import numpy as np
import requests
from PyPDF2 import PdfReader
from sentence_transformers import SentenceTransformer
from transformers import pipeline

# ------------------------------
# 1. Download PDF from URL
# ------------------------------
url = "https://arxiv.org/pdf/1706.03762.pdf"  # You can change this URL
pdf_path = "/content/sample.pdf"

response = requests.get(url)
with open(pdf_path, "wb") as f:
    f.write(response.content)

print("PDF downloaded successfully!")

# ------------------------------
# 2. Load PDF
# ------------------------------
def load_pdf(file_path):
    reader = PdfReader(file_path)
    text = ""
```

Toggle Gemini

ex.no 07 in generative ai

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```
Setting `pad_token_id` to `eos_token_id`:50256 for open-end generation.
Both `max_new_tokens` (=256) and `max_length`(=200) seem to have been set. `max_new_tokens` will take precedence. Please refer to the documentation for more information. (ht
Setting `pad_token_id` to `eos_token_id`:50256 for open-end generation.
Both `max_new_tokens` (=256) and `max_length`(=200) seem to have been set. `max_new_tokens` will take precedence. Please refer to the documentation for more information. (ht

--- ANSWER ---

    Answer based on the context:

    and Beatrice Santorini. Building a large annotated corpus of english: The penn treebank. Computational linguistics , 19(2):313–330, 1993. [26] David McClosky, Eugene Cha
Provided proper attribution is provided, Google hereby grants permission to reproduce the tables and figures in this paper solely for use in journalistic or scholarly works.
is what we are missing , in my opinion . <EOS> <pad> The Law will never be perfect , but its application should be just - this is what we are missing , in my opinion . <EOS>

    Question: What is the main idea of the paper?
    Answer:
     2016. doi:10.1537/a005085 [29] Paulus, Caiming, and S. P. Bruecken. A neural network for semantic processing. arXiv preprint arXiv:1705.04305, 2016. doi:10.1537/a005085

--- ANSWER ---

    Answer based on the context:

    3.2.3 Applications of Attention in our Model The Transformer uses multi-head attention in three different ways: •In "encoder-decoder attention" layers, the queries come
positions, since for any fixed offset k,PEpos+kcan be represented as a linear function of PEpos. We also experimented with using learned positional embeddings [ 9] instead,
is what we are missing , in my opinion . <EOS> <pad> The Law will never be perfect , but its application should be just - this is what we are missing , in my opinion . <EOS>

    Question: Explain attention mechanism
    Answer:
     layers are used in the main model. The encoder is a special case of a "decoder-decoder" that is a "decoder-decode" that is the result of a "decoder-decode" in the main
```